

SPAM MESSAGE DETECTION USING NLP AND MACHINE LEARNING

Mini NLP Project Report

Course	Natural Language Processing
Project	Spam Message Detection Using NLP and Machine Learning
Platform	Google Colab
Programming Language	Python

1. Title

Spam Message Detection Using NLP and Machine Learning

2. Problem Statement

Spam messages are unwanted messages that may contain advertisements, fake offers, scams, or irrelevant content. With the increasing use of SMS and mobile communication, users receive many messages, making it difficult to identify spam messages manually.

The goal of this project is to develop an NLP-based system that automatically classifies SMS messages into two categories: Spam and Ham (Not Spam).

The system analyzes the text content of an SMS message and uses Natural Language Processing and Machine Learning techniques to predict its category.

3. Objectives

The main objectives of this project are:

- To understand the application of Natural Language Processing in text classification.
- To study and analyze an SMS spam dataset.
- To preprocess and clean SMS text data.
- To convert text into numerical features using TF-IDF.
- To train a machine learning classification model.
- To classify messages as Spam or Ham.
- To evaluate the performance of the model.
- To test the model using new SMS messages.

4. Dataset

4.1 Dataset Name

SMS Spam Collection Dataset

4.2 Dataset Source

UCI Machine Learning Repository — SMS Spam Collection Dataset

4.3 Dataset Description

The SMS Spam Collection dataset contains SMS messages labelled as either Ham or Spam.

- **Ham:** Normal or legitimate SMS messages.
- **Spam:** Unwanted or suspicious SMS messages.

The dataset contains 5,574 SMS messages.

4.4 Dataset Attributes

Attribute	Description
Label	Indicates whether the message is Ham or Spam
Message	Contains the actual SMS text

4.5 Target Variable

The target variable is Label, which contains two classes:

- Ham
- Spam

4.6 Data Preparation

Before training the model, the SMS messages are cleaned and prepared for analysis. The preprocessing includes:

- Converting text to lowercase.
- Removing unnecessary characters.
- Removing extra spaces.
- Preparing the cleaned text for feature extraction.

After preprocessing, the text is converted into numerical features using TF-IDF.

5. Methodology

The project follows the following NLP pipeline:

SMS Dataset → Text Preprocessing → TF-IDF Feature Extraction → Train-Test Split → Multinomial Naive Bayes Classification → Prediction → Evaluation

5.1 Text Preprocessing

Raw SMS messages cannot be directly used by a traditional machine learning model. Therefore, the messages are cleaned before classification. The following preprocessing steps are performed:

- Convert all text to lowercase.

- Remove unnecessary characters.
- Remove extra spaces.
- Prepare the cleaned text for feature extraction.

5.2 TF-IDF Feature Extraction

TF-IDF stands for Term Frequency-Inverse Document Frequency. TF-IDF is used to convert text messages into numerical feature vectors that can be processed by the machine learning model. It assigns importance to words based on their occurrence in individual messages and across the collection of messages.

5.3 Train-Test Split

The dataset is divided into two parts:

- 80% Training Data
- 20% Testing Data

The training data is used to train the model, while the testing data is used to evaluate its performance on unseen messages.

5.4 Classification

The project uses the Multinomial Naive Bayes algorithm. The model learns patterns from the training messages and predicts whether new messages are Spam or Ham.

6. Implementation

6.1 Programming Language

Python

6.2 Development Platform

Google Colab is used to write, execute, and test the Python code.

6.3 Libraries Used

The following Python libraries are used:

- **Pandas** — To load and manage the dataset.
- **NumPy** — For numerical calculations.
- **Matplotlib** — To create graphs.
- **Seaborn** — For data visualization and the confusion matrix.
- **Scikit-learn** — For TF-IDF, machine learning, model training, and evaluation.
- **Regular Expressions (re)** — To clean and preprocess the text.

6.4 NLP Technique

TF-IDF Vectorization

6.5 Machine Learning Algorithm

Multinomial Naive Bayes

6.6 Implementation Process

The following steps are followed:

- 1 Import the required libraries.
- 2 Load the SMS dataset.
- 3 Explore the dataset.
- 4 Check for missing values.
- 5 Convert Spam and Ham labels into numerical values.
- 6 Clean and preprocess the SMS messages.
- 7 Split the data into training and testing sets.
- 8 Apply TF-IDF feature extraction.
- 9 Train the Multinomial Naive Bayes model.
- 10 Make predictions on the test data.
- 11 Calculate evaluation metrics.
- 12 Generate the confusion matrix.
- 13 Test the model using new SMS messages.

7. Results

The trained model was evaluated using Accuracy, Precision, Recall, F1-Score, and a Confusion Matrix.

7.1 Performance Metrics

The model achieved the following results:

Metric	Result
Accuracy	96.59%
Precision	100.00%
Recall	74.50%
F1-Score	85.38%

The model achieved an accuracy of 96.59%, showing that it correctly classified most of the test messages.

The precision of 100% indicates that the messages predicted as Spam by the model were correctly classified as Spam in the test results.

The recall of 74.50% indicates that the model identified a significant portion of the actual Spam messages, although some Spam messages were missed.

The F1-Score of 85.38% represents the balance between precision and recall.

7.2 Confusion Matrix

A confusion matrix is used to show the number of messages correctly and incorrectly classified as Ham and Spam.

Figure 1: Confusion Matrix of the Spam Detection Model

7.3 Model Performance Graph

A performance graph is used to compare the Accuracy, Precision, Recall, and F1-Score of the model.

Figure 2: Model Performance Graph

7.4 Sample Prediction

The trained model can classify new SMS messages entered by the user.

Example 1

Input	Congratulations! You have won a free prize. Call now!
Output	Prediction: SPAM

Example 2

Input	Hey, are you coming to class tomorrow?
-------	--

Output	Prediction: HAM
---------------	-----------------

Figure 3: Sample SMS Prediction

8. Limitations

The project has the following limitations:

- The model's performance depends on the size and quality of the dataset.
- It may incorrectly classify some messages that it has not seen before.
- Sarcasm and unusual language can be difficult for the model to detect.
- New types of spam messages may not always be identified correctly.
- The model is primarily designed for SMS text classification.
- The text preprocessing method used in this project is relatively basic.

9. Future Scope

The project can be improved in the future by:

- Using a larger and more diverse dataset.
- Comparing multiple machine learning algorithms.
- Improving the text preprocessing techniques.
- Using advanced NLP models such as BERT and other Transformer-based models.
- Adding multilingual support.
- Developing a real-time spam detection system.
- Deploying the model as a web application.
- Developing a mobile application for real-time SMS classification.
- Continuously updating the model using new spam examples.

10. Conclusion

This project developed an NLP-based SMS Spam Detection system using text preprocessing, TF-IDF feature extraction, and the Multinomial Naive Bayes machine learning algorithm.

The system processes SMS messages and classifies them into two categories: Spam and Ham. The model achieved an accuracy of 96.59%, with 100% precision, 74.50% recall, and an F1-Score of 85.38% on the test data.

The results demonstrate that TF-IDF combined with Multinomial Naive Bayes can be used effectively for SMS spam classification.

Through this project, concepts such as text preprocessing, feature extraction, machine learning classification, model evaluation, and practical NLP application development were explored.

The system can be further improved by using larger datasets, advanced NLP techniques, multilingual support, and real-time deployment.